

## Concepts and questions

1. Scope in js ( Global, functional, block )
2. Scope chaining
3. Primitive , non-primitive in js
4. Var, let and const
5. Temporal dead zone
6. Hoisting
7. Prototypes in js
8. Prototype object
9. Prototype chaining
10. Closures
11. Pass by reference vs pass by value
12. Currying in js
13. Infinite currying problem
14. Memoization in js
15. Rest parameter
16. Spread operators
17. How many ways you can create object in js
18. Generator functions in js
19. Javascript single threaded language
20. Callbacks, why we need
21. Event loops
22. Callback queue
23. Microtask queue
24. Promises
25. Event propagation
26. Event bubbling
27. Event capturing
28. Event stop propagation
29. Event delegation
30. Type coercion vs conversion in js
31. Throttle
32. Debounce
33. How javascript parse and compile your code, step by step
34. If a microtask has a function which continuously puts its return function in the microtask queue infinite, how to handle this ?
35. First class functions in js
36. Immediate invoke functions
37. Call, apply and bind, why do we use this
38. Maplimit
39. Async and await

40. Then and catch
41. Variable shadowing
42. What does static mean in a JavaScript class?
43. Undefined vs not-defined, null in js
44. High order functions
45. Callback hell
46. This concept in js
47. Function declaration, expression, anonymous functions, arrow functions
48. Async vs defer
49. Execution context
50. Callstack
51. Garbage collection
52. Equality ( === vs == )
53. Strict mode
54. Lexical env
55. Explain Temporal Dead Zone . Why does typeof x throw error for let x before declaration but not for undeclared variables? ( updated )
56. Explain execution context creation phase and execution phase. What gets stored in lexical environment? ( updated )
57. Explain lexical scope vs dynamic scope. Does JavaScript use lexical or dynamic scoping? ( updated )
58. Explain equality comparison: ==, ===, Object.is, SameValueZero. Why is NaN === NaN false but Object.is(NaN, NaN) true?
59. Explain prototype pollution. How can unsafe object merge logic become a security issue?

## **Polyfills-**

1. Map, reduce, filter, forEach, find
2. Call, apply, bind
3. Promise, Promise.all(), Promise.any(), Promise.allSettled(), Promise.race()
4. Maplimit
5. Debounce
6. Throttle
7. Event emitter
8. setInterval polyfill
9. Parallel limit function
10. Deep vs shallow copy
11. Flatten deeply nested object
12. Memoization/ caching
13. Promise -> finally

14. retry

### **Best Video/practice links-**

1. Practice js coding question- [Leetcode 30 days of JS](#) ( VIMP )
2. Playlist for polyfills and questions - [Road side coder](#)
3. Playlist to understand each concept of js - [Namaste javascript](#)
4. Documentation, fundamentals, basics, dataTypes - [Javascript info](#)
5. Learn with curiosity and ask chatgpt to help in understanding depth